Multi-threaded Web Server User Manual

Made by: Bernardo Reis 126004 | Fernando Santos 124808

In this document, we will explain how to use, debug, test, and add new features to the program.

Requirements

- Operating System: Linux (Ubuntu 20.04+ or similar)
- Compiler: GCC 9.0+ with C11 support
- Libraries: - POSIX threads (pthread)
- POSIX real-time extensions (rt)
- Standard C library
- Needed Linux Commands (some Distros don't have):
    - GNU make
    - Logrotate
    - apachebench (for debugging)
    - Valgrind

Installation may vary from distro to distro.

Relevant commands

To run the server, use **./server** in the project folder. To implement changes, write **make** in the terminal in the project folder.

There are also other important commands that should ran with the server open:

- ab -n 1000 -c 10 -k https://localhost:8080/ (Debugging, low-load)
- ab -n 10000 -c 100 -k https://localhost:8080/ (Debugging, high-load)
- valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./server (Debugging, memory leaks)

- valgrind --tool=helgrind --log-file=helgrind.log ./server (Debugging, race conditions)
- time curl -s https://localhost:8080/ > /dev/null (For cache miss testing)
- For cache hit testing do:

```
for i in {1..10}; do
    time curl -s https://localhost:8080/ > /dev/null
done
```

The *-k* in ApacheBench commands is optional and is used to test keep-alive connections.

Configuration File

In the project folder there is a file named “server.conf” that contains all relevant global variables that can (and should) be adapted to your needs and infrastructure.

Log File

It captures a chronological history of incoming requests, error messages, and system events, serving as the primary resource for monitoring server performance and debugging potential issues. To see older logs, check the old_logs folder, the older it is, the bigger number it has.

User Guidance

In this chapter, we will explain how to check all our debug endpoints on the website. Firstly, when navigating to the root of the website (https://localhost:8080/), it will return the index.html file, which can be found in the www folder. All the server files are by default, inside that folder. The index.html file can also be returned by the server if the request path is “/index.html” (https://localhost:8080/index.html). To check the statistics dashboard, please go to “/statistics.html” or check the terminal every 30

seconds. To show that the server supports different types of files, we also provided some test files or paths that can be checked for debugging:

- “/miau.png” to check Content-Type image/png
- “/lusiadas.pdf” to check Content-Type application/pdf
- “/admin” to display the Unauthorized Error (401)
- anything other than what is inside www will result in Page Not Found error (404)
- As requested in the “Announcement of Project Proposal” PDF, index.html is served for directory requests, so, for example, “/errors/” or “/errors” will return the index.html.

To check, for example, status code 403, 500 or 503 the process is more difficult. For 500, the best way is temporarily removing the “notfound.html” file. For 403, we recommend this test:

```
echo "test" > "../www/forbidden_test.txt"
chmod 000 "../www/forbidden_test.txt"
curl -k -I "https://localhost:8080/forbidden_test.txt" 2>&1 | grep -q "403 Forbidden"
assert_success "Status 403"
rm -f "../www/forbidden_test.txt"
```

For 503, this test should do the trick:

```
ab -n 1000 -c 250 -k https://localhost:8080/lusiadas.pdf > /dev/null 2>&1 &
AB_PID=$!
sleep 0.3
for i in {1..5}; do
  curl -k -I "https://localhost:8080/" 2>&1 | grep -q "503 Service Unavailable" && break
  sleep 0.1
done
echo "$?"
```

If the returned integer was 0, the error was not achieved. If it was 1, the error was achieved.

Small advice: The command above starts a background process, (ab with &), so after a while, please make sure the program terminates correctly with the command “ps”. If the command returned a table that contained a line with “/server” and you have already checked the result, please do the command “pkill server”.

It’s also important to refer to the script full_testing.sh, inside the folder “tests”. That script executes all the debugging tests to check the server's status.

Bernardo Reis | 126004
Fernando Santos | 124808